

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE NAME: Database Management Systems

COURSE CODE: CSA05

COURSE OUTCOME COVERED

CO2: *Apply the relational model, relational algebra, SQL query structures, and views to solve database querying and data manipulation problems. (BL3, BL6)*

Activity Tool : Scenario-Based Task (High)

Assessment Tool Weightage: 35%

Dr.G.Ayyappan

QUESTIONS			Total Marks: 50
Q.No	Question	Bloom's Level	Marks
1	Given the scenario of a School Management System, identify relations and write SQL to list all students with their class teacher and grade.	BL3 – Apply	5
2	A hospital wants to track patient appointments. Design the relational schema and write SQL to find doctors with more than 10 appointments this month.	BL3 – Apply	5
3	In an e-commerce scenario, write SQL queries using sub-queries to find products that have never been ordered.	BL3 – Apply	5
4	For a university database, write relational algebra expressions to find students enrolled in both 'DBMS' and 'OS' courses.	BL3 – Apply	5
5	Given a payroll scenario, create a view showing employees whose salary is below department average and write a query to update their salary by 10%.	BL6 – Create	5
6	In a banking scenario, apply JOIN operations to retrieve customer account details, transaction history, and branch information.	BL3 – Apply	5

7	A retail store needs daily sales summary. Write SQL with GROUP BY and HAVING to report total sales per category exceeding Rs. 10,000.	BL3 – Apply	5
8	For a library system scenario, construct tuple relational calculus expressions to find members who borrowed books from all genres.	BL3 – Apply	5
9	Given an inventory management scenario, define CHECK and FOREIGN KEY constraints and demonstrate their enforcement through SQL.	BL3 – Apply	5
10	Design a relational schema for an HR system and create materialized views for monthly attendance and performance summaries.	BL6 – Create	5

Code of Conduct:

I E B . A S H W I N I / 1 9 2 5 1 1 4 2 2 _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:
Ashwini

RUBRICS FOR CALCULATION:

Scenario based assessment					
Criteria	Weightage (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Understanding of Scenario	20%	Demonstrates clear and complete understanding of the scenario and context	Shows good understanding with minor gaps	Partial understanding ; misses key aspects	Misinterprets or fails to understand the scenario
Identification of Key Issues	20%	Accurately identifies all relevant issues and factors involved	Identifies most key issues with minor omissions	Identifies limited issues; some irrelevant points	Unable to identify key issues
Application of Concepts	25%	Applies appropriate concepts effectively to	Applies concepts with minor errors	Limited or partially correct application	Unable to apply relevant concepts

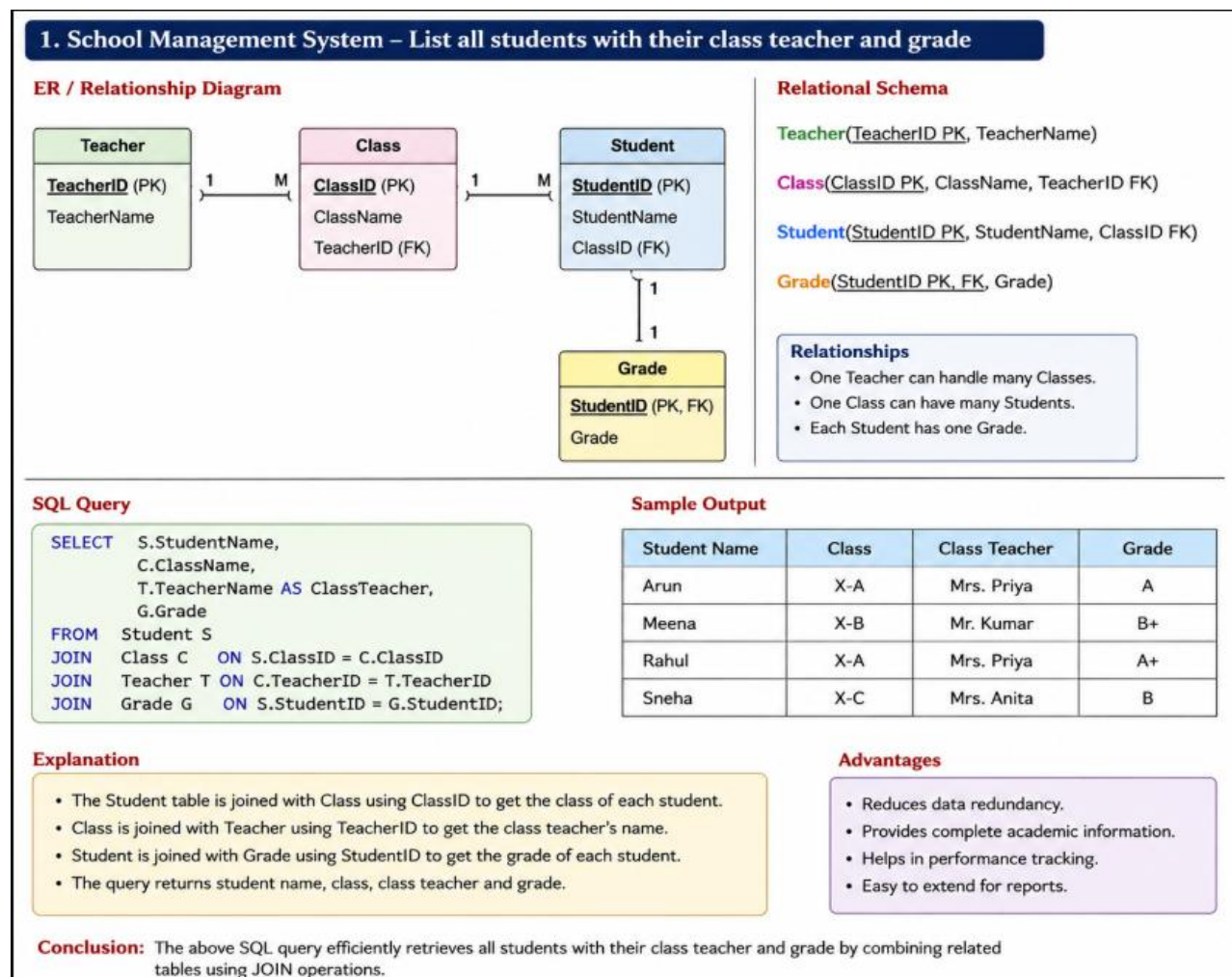
		address the scenario			
Decision Making	20%	Makes well-reasoned, logical decisions with strong rationale	Makes reasonable decisions with some justification	Makes weak decisions with limited justification	No clear decisions made or rationale provided
Clarity of Response	15%	Response is clear, structured, and logically presented	Generally clear with minor issues	Somewhat unclear or poorly structured	Disorganized and difficult to understand

1. Given the scenario of a School Management System, identify relations and write SQL to list all students with their class teacher and grade.

School Management System:

Scenario Understanding

A School Management System is used to manage information related to students, classes, teachers, and academic performance. Each student belongs to a specific class, and every class is assigned a class teacher. The system also stores the grades obtained by students. The requirement is to identify the relationships between these tables and write an SQL query to display each student's name, class, class teacher, and grade. This report helps teachers, administrators, and parents monitor student progress and maintain organized academic records.



Identification of Key Issues

- Store student details without duplication.
- Maintain the relationship between students, classes, teachers, and grades.
- Retrieve complete student information using SQL JOIN operations.
- Ensure that the report is accurate and easy to understand.

Relations (Schema)

Student (StudentID, StudentName, ClassID)

Class (ClassID, ClassName, TeacherID)

Teacher (TeacherID, TeacherName)

Grade (StudentID, Grade)

Relationship

- One class can have **many students**.
- One teacher can manage **one or more classes**.
- Each student has **one grade record**.

SQL Query

```
SELECT S.StudentName,  
       C.ClassName,  
       T.TeacherName AS ClassTeacher,  
       G.Grade  
FROM Student S  
JOIN Class C  
ON S.ClassID = C.ClassID  
JOIN Teacher T  
ON C.TeacherID = T.TeacherID  
JOIN Grade G  
ON S.StudentID = G.StudentID;
```

Explanation

The query begins with the **Student** table because it contains the student records. It is joined with the **Class** table using **ClassID** to identify the class of each student. The **Teacher** table is then joined using **TeacherID** to retrieve the class teacher's name. Finally, the **Grade** table is joined using **StudentID** to display the student's academic grade. Using JOIN operations avoids data duplication and combines related information from different tables into a single report.

Sample Output

Student Name	Class	Class Teacher	Grade
Arun	X-A	Mrs. Priya	A
Meena	X-B	Mr. Kumar	B+
Rahul	X-A	Mrs. Priya	A+

Advantages

- Retrieves complete student information in a single query.

- Reduces data redundancy by using relational tables.
- Makes academic reporting faster and more accurate.
- Supports better decision-making for teachers and school administrators.

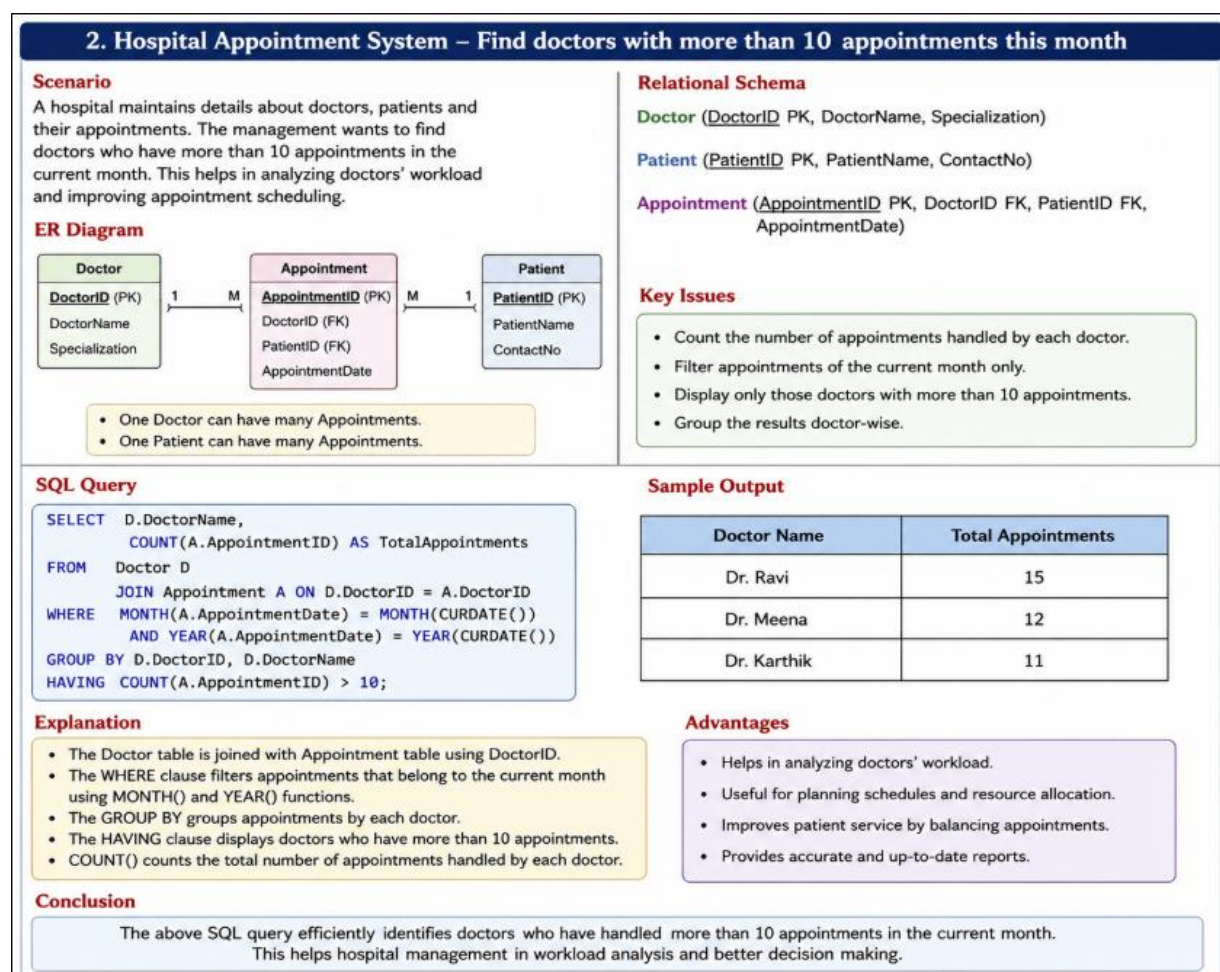
The School Management System uses relational tables and SQL JOIN operations to efficiently combine student, class, teacher, and grade information. This approach ensures accurate record management, improves data consistency, and generates meaningful reports that help monitor students' academic performance.

2.A hospital wants to track patient appointments. Design the relational schema and write SQL to find doctors with more than 10 appointments this month.

Hospital Appointment System

Scenario Understanding

A hospital management system stores details about doctors, patients, and their appointments. Every appointment is assigned to a doctor and a patient on a particular date. The hospital administration wants to identify doctors who have attended **more than 10 appointments in the current month**. This helps in analyzing the doctors' workload, improving appointment scheduling, and ensuring quality healthcare services.



Identification of Key Issues

- Maintain records of doctors, patients, and appointments.
- Count the number of appointments handled by each doctor.
- Identify doctors with more than 10 appointments in the current month.
- Generate a report for hospital management.

Relational Schema

Doctor (DoctorID, DoctorName, Specialization)

Patient (PatientID, PatientName)

Appointment (AppointmentID, DoctorID, PatientID, AppointmentDate)

Relationship

- One doctor can attend **many appointments**.
- One patient can have **many appointments**.
- Each appointment is linked to **one doctor and one patient**.

SQL Query

```
SELECT D.DoctorName,  
       COUNT(A.AppointmentID) AS TotalAppointments  
FROM Doctor D  
JOIN Appointment A  
ON D.DoctorID = A.DoctorID  
WHERE MONTH(A.AppointmentDate) = MONTH(CURDATE())  
AND YEAR(A.AppointmentDate) = YEAR(CURDATE())  
GROUP BY D.DoctorID, D.DoctorName  
HAVING COUNT(A.AppointmentID) > 10;
```

Explanation

The query joins the **Doctor** and **Appointment** tables using **DoctorID**. It filters appointments that belong to the current month using the MONTH() and YEAR() functions. The GROUP BY clause groups appointments by doctor, while the COUNT() function calculates the total appointments handled by each doctor. Finally, the HAVING clause displays only those doctors who have attended more than 10 appointments.

Sample Output

Doctor Name	Total Appointments
Dr. Ravi	15
Dr. Meena	12

Advantages

- Helps balance doctors' workload.
- Improves appointment scheduling.
- Supports hospital management in resource planning.
- Generates accurate monthly reports.

The SQL query efficiently identifies doctors with heavy workloads during the current month, helping the hospital improve patient care and staff management.

3. In an e-commerce scenario, write SQL queries using sub-queries to find products that have never been ordered

E-Commerce System

Scenario Understanding

An e-commerce company maintains information about products and customer orders. Some products may never be purchased by customers. The company wants to identify such products so that discounts, advertisements, or promotional offers can be introduced to improve sales.

Identification of Key Issues

- Store product and order information separately.
- Compare ordered and unordered products.
- Use a subquery to identify products that have never been sold.
- Improve inventory management.

Relational Schema

Product (ProductID, ProductName, Price)

Orders (OrderID, ProductID, OrderDate)

Relationship

- One product can appear in many orders.
- Some products may not appear in any order.

SQL Query

```
SELECT ProductName
FROM Product
WHERE ProductID NOT IN
(
SELECT ProductID
FROM Orders
);
```

Explanation

The subquery retrieves all Product IDs that exist in the **Orders** table. The main query compares these IDs with the **Product** table and returns only those products that are not present in any order. The NOT IN operator is used to identify products that have never been purchased.

Sample Output

Product Name
Smart Watch
Bluetooth Speaker
Wireless Keyboard

Advantages

- Identifies slow-moving products.
- Supports inventory planning.
- Helps improve marketing strategies.
- Reduces storage costs.

Using a subquery, the e-commerce company can quickly identify products that have never been ordered and take suitable business decisions to increase sales.

4. For a university database, write relational algebra expressions to find students enrolled in both 'DBMS' and 'OS' courses.

Scenario Understanding

A university database stores student enrollment information for different courses. The university wants to identify students who are enrolled in both **DBMS** and **Operating System (OS)** courses. This information is useful for preparing class lists, scheduling laboratory sessions, and monitoring academic progress.

Identification of Key Issues

- Maintain student and course records.
- Store enrollment details.
- Find students enrolled in both specified courses.
- Use Relational Algebra for the solution.

Relational Schema

Student (StudentID, StudentName)

Course (CourseID, CourseName)

Enrollment (StudentID, CourseID)

Relationship

- One student can enroll in many courses.
- One course can have many students.

Relational Algebra

π StudentID

$(\sigma \text{ CourseName} = \text{'DBMS'})$

$(\text{Student} \bowtie \text{Enrollment} \bowtie \text{Course})$

$\cap$

π StudentID

$(\sigma \text{ CourseName} = \text{'OS'})$

$(\text{Student} \bowtie \text{Enrollment} \bowtie \text{Course})$

Explanation

The first expression retrieves students enrolled in **DBMS**. The second expression retrieves students enrolled in **OS**. The intersection ($\cap$) operator returns only those students who are common in both result sets.

Sample Output

Student ID	Student Name
101	Arun
115	Meena

Advantages

- Efficiently identifies common enrollments.
- Helps in timetable planning.
- Useful for course coordination.
- Reduces manual searching.

Relational Algebra provides a simple and effective method for identifying students enrolled in both DBMS and OS courses.

5. Given a payroll scenario, create a view showing employees whose salary is below department average and write a query to update their salary by 10%.

Scenario Understanding

A company maintains employee salary details for different departments. The management wants to identify employees whose salary is below the average salary of their department. These employees should receive a **10% salary increment** to maintain salary fairness and improve employee satisfaction.

Identification of Key Issues

- Calculate the department-wise average salary.
- Identify employees earning below the average.
- Create a view for easy access.
- Update salaries by 10%.

Relational Schema

Employee (EmpID, EmpName, DeptID, Salary)

Relationship

- One department can have many employees.
- Each employee belongs to one department.

View Creation

```
CREATE VIEW LowSalaryEmployees AS
```

```
SELECT *
```

```
FROM Employee E
```

```
WHERE Salary <
```

```
(
```

```
SELECT AVG(Salary)
```

```
FROM Employee
```

```
WHERE DeptID = E.DeptID
```

```
);
```

Salary Update

```
UPDATE Employee
```

```
SET Salary = Salary * 1.10
```

```
WHERE EmpID IN
```

```
(
```

```
SELECT EmpID
```

```
FROM LowSalaryEmployees
```

```
);
```

Explanation

The view stores employees whose salaries are lower than the average salary of their department. The update statement increases their salary by **10%** using the view. This approach improves readability and avoids writing the same query repeatedly.

Sample Output

Before Update

Employee	Salary
Arun	28,000
Meena	30,000

After Update

Employee	Salary
Arun	30,800
Meena	33,000

Advantages

- Ensures fair salary distribution.
- Simplifies repeated salary analysis using a view.
- Improves employee motivation.
- Supports payroll management.

Creating a view and updating salaries based on department averages helps maintain salary equality and improves the efficiency of the payroll management system.

6. In a banking scenario, apply JOIN operations to retrieve customer account details, transaction history, and branch information

Scenario Understanding

A banking system maintains information about customers, their bank accounts, transactions, and branch details. Customers may have one or more accounts, and every account belongs to a specific branch. The bank wants to retrieve customer account details along with transaction history and branch information. This report helps bank employees monitor customer activities, verify transactions, and provide better customer service.

6. Banking System – Retrieve customer account details, transaction history and branch information

Scenario Understanding

A bank maintains data about customers, their accounts, transactions performed on the accounts and the branches where the accounts are maintained. The requirement is to retrieve customer details along with their account details, transaction history and the branch information.

ER Diagram

```
graph LR
    Customer[Customer] -- "1:M" --> Account[Account]
    Account -- "1:M" --> Transaction[Transaction]
    Transaction -- "M:1" --> Branch[Branch]
```

- One Customer can have many Accounts.
- One Account is maintained in one Branch.
- One Account can have many Transactions.
- One Branch can manage many Accounts.

Relational Schema

Customer (CustomerID PK, CustomerName)

Account (AccountNo PK, CustomerID FK, BranchID FK)

Transaction (TransactionID PK, AccountNo FK, Amount, TransactionDate)

Branch (BranchID PK, BranchName, BranchLocation)

Key Points

- ✓ Link all tables using Primary and Foreign Keys.
- ✓ Use JOIN operations to combine related data.
- ✓ Display customer, account, transaction and branch details in a single result.
- ✓ Helps in monitoring customer activities and transactions.

SQL Query

```
SELECT C.CustomerName,
       A.AccountNo,
       T.TransactionID,
       T.Amount,
       T.TransactionDate,
       B.BranchName
FROM Customer C
JOIN Account A ON C.CustomerID = A.CustomerID
JOIN Transaction T ON A.AccountNo = T.AccountNo
JOIN Branch B ON A.BranchID = B.BranchID
ORDER BY C.CustomerName, T.TransactionDate;
```

Sample Output

Customer Name	Account No	Transaction ID	Amount (₹)	Transaction Date	Branch Name
Arun	100245	T101	5,000.00	01-05-2024	Chennai Main
Arun	100245	T102	2,000.00	05-05-2024	Chennai Main
Meena	100356	T108	2,500.00	03-05-2024	Coimbatore
Meena	100356	T109	1,200.00	07-05-2024	Coimbatore
Rahul	100789	T115	3,800.00	02-05-2024	Madurai

Explanation

- The query starts from Customer table (C) to get customer details.
- It joins Account table (A) using CustomerID to get account information.
- It joins Transaction table (T) using AccountNo to fetch transaction history.
- It joins Branch table (B) using BranchID to get branch details.
- ORDER BY is used to sort the result by customer name and transaction date.

Advantages

- Provides complete banking information in one view.
- Helps in tracking customer transactions easily.
- Useful for account statements, auditing and reporting.
- Improves customer service and decision making.

Conclusion

The above SQL query uses JOIN operations to retrieve customer account details, transaction history and branch details efficiently. This helps the bank in monitoring activities and maintaining accurate records.

Identification of Key Issues

- Store customer, account, transaction, and branch information.
- Link all tables using primary and foreign keys.
- Retrieve complete banking information using JOIN operations.
- Generate a report for customer account analysis.

Relational Schema

Customer (CustomerID, CustomerName)

Account (AccountNo, CustomerID, BranchID)

Transaction (TransactionID, AccountNo, Amount, TransactionDate)

Branch (BranchID, BranchName, BranchLocation)

Relationship

- One customer can have many accounts.
- One account belongs to one branch.
- One account can have many transactions.
- One branch manages many accounts.

SQL Query

```
SELECT C.CustomerName,  
       A.AccountNo,  
       T.TransactionID,  
       T.Amount,  
       T.TransactionDate,  
       B.BranchName  
FROM Customer C  
JOIN Account A  
ON C.CustomerID = A.CustomerID  
JOIN Transaction T  
ON A.AccountNo = T.AccountNo  
JOIN Branch B  
ON A.BranchID = B.BranchID;
```

Explanation

The query begins with the **Customer** table and joins it with the **Account** table using **CustomerID**. Next, the **Transaction** table is joined using **AccountNo** to display transaction history. Finally, the **Branch** table is joined using **BranchID** to show the branch where the account is maintained. SQL JOIN operations combine related records into one comprehensive report.

Sample Output

Customer	Account No	Transaction ID	Amount	Branch
Arun	100245	T101	₹5,000	Chennai
Meena	100356	T108	₹2,500	Coimbatore

Advantages

- Retrieves complete banking information in one query.
- Improves customer service.
- Helps monitor transactions.
- Supports banking reports and auditing.

JOIN operations effectively combine customer, account, transaction, and branch details, enabling banks to maintain accurate records and provide efficient financial services.

7.A retail store needs daily sales summary. Write SQL with GROUP BY and HAVING to report total sales per category exceeding Rs. 10,000.

Scenario Understanding

A retail store records daily sales for different product categories such as Electronics, Grocery, Clothing, and Furniture. The management wants to generate a daily sales summary showing only those categories whose total sales exceed **₹10,000**. This helps identify high-performing product categories and supports business decision-making.

Identification of Key Issues

- Store category-wise daily sales.
- Calculate the total sales for each category.
- Display only categories with sales greater than ₹10,000.
- Use **GROUP BY** and **HAVING** clauses.

Relational Schema

Sales (SaleID, Category, Amount, SaleDate)

Relationship

Each sale belongs to one product category, and one category can have many sales records.

SQL Query

```
SELECT Category,  
       SUM(Amount) AS TotalSales  
FROM Sales  
WHERE SaleDate = CURDATE()  
GROUP BY Category  
HAVING SUM(Amount) > 10000;
```

Explanation

The **WHERE** clause filters sales made on the current date. The **GROUP BY** clause groups records according to product category. The **SUM()** function calculates the total sales for each category. Finally, the **HAVING** clause displays only those categories whose total sales exceed ₹10,000. This provides a summarized daily sales report.

Sample Output

Category	Total Sales
Electronics	₹18,500
Grocery	₹12,400

Advantages

- Generates category-wise sales reports.

- Helps identify profitable categories.
- Supports inventory planning.
- Improves business analysis and decision-making.

The SQL query provides an efficient method for generating daily sales summaries, helping retail managers monitor business performance and make informed decisions.

8. For a library system scenario, construct tuple relational calculus expressions to find members who borrowed books from all genres.

Scenario Understanding

A library maintains records of members, books, genres, and borrowing details. The librarian wants to identify members who have borrowed books from **every available genre**. This information can be used to recognize active readers, recommend books, or reward members with reading achievements.

Identification of Key Issues

- Maintain records of members and books.
- Store borrowing history.
- Check whether a member has borrowed books from all genres.
- Use Tuple Relational Calculus (TRC).

Relational Schema

Member (MemberID, MemberName)

Book (BookID, Genre)

Borrow (MemberID, BookID)

Relationship

- One member can borrow many books.
- One book belongs to one genre.
- Members can borrow books from different genres.

Tuple Relational Calculus

$$\{ M \mid$$
$$\text{Member}(M)$$
$$\wedge$$
$$\forall G$$
$$(\text{Genre}(G)$$
$$\rightarrow$$
$$\exists B \exists R$$
$$(\text{Book}(B)$$
$$\wedge B.\text{Genre} = G$$
$$\wedge \text{Borrow}(R)$$
$$\wedge R.\text{MemberID} = M.\text{MemberID}$$
$$\wedge R.\text{BookID} = B.\text{BookID}))$$
$$\}$$

Explanation

The TRC expression checks every genre available in the library. For each genre, there must exist at least one borrowed book belonging to that genre by the same member. Only members satisfying this condition are included in the result.

Sample Output

Member ID	Member Name
M101	Arun
M115	Priya

Advantages

- Identifies active readers.
- Helps in issuing reading awards.
- Supports personalized book recommendations.
- Demonstrates universal quantification in TRC.

Tuple Relational Calculus provides a logical and effective way to identify members who have borrowed books from all available genres.

9. Given an inventory management scenario, define CHECK and FOREIGN KEY constraints and demonstrate their enforcement through SQL.

Scenario Understanding

An inventory management system stores details about products and suppliers. To maintain data accuracy, constraints such as **CHECK** and **FOREIGN KEY** are used. These constraints prevent

9. Inventory Management System – Demonstrate CHECK and FOREIGN KEY Constraints

Scenario Understanding

An inventory management system stores details about products and suppliers. To maintain data accuracy and integrity, CHECK and FOREIGN KEY constraints are used. The CHECK constraint ensures valid product prices, and the FOREIGN KEY constraint ensures that every product is linked to an existing supplier.

ER Diagram

- One Supplier can supply many Products.
- Each Product is supplied by one Supplier.
- SupplierID in Product is a Foreign Key referencing Supplier.

Relational Schema

Supplier (SupplierID PK, SupplierName)
Product (ProductID PK, ProductName, Price, SupplierID FK)

SQL Statements

```
CREATE TABLE Supplier
(
  SupplierID INT PRIMARY KEY,
  SupplierName VARCHAR(50) NOT NULL
);

CREATE TABLE Product
(
  ProductID INT PRIMARY KEY,
  ProductName VARCHAR(50) NOT NULL,
  Price DECIMAL(10,2) CHECK (Price > 0),
  SupplierID INT,
  FOREIGN KEY (SupplierID)
  REFERENCES Supplier(SupplierID)
);
```

Constraint Demonstration

1. Violation of CHECK Constraint

```
INSERT INTO Product
VALUES (101, 'Mouse', -500, 1);
```

Result:

CHECK constraint violated:
Price must be greater than 0.

2. Violation of FOREIGN KEY Constraint

```
INSERT INTO Product
VALUES (102, 'Keyboard', 700, 99);
```

Result:

FOREIGN KEY constraint violated:
SupplierID 99 does not exist in Supplier table.

Explanation

- The CHECK constraint in Product table ensures that Price is always greater than zero.
- The FOREIGN KEY constraint in Product table ensures that SupplierID must exist in the Supplier table.
- If we try to insert invalid data, the database rejects the operation and maintains data integrity.

Advantages

- ✓ Prevents invalid or incorrect data entry.
- ✓ Maintains referential integrity between tables.
- ✓ Improves reliability and consistency of the database.
- ✓ Reduces data redundancy and manual errors.

Conclusion

CHECK and FOREIGN KEY constraints play a crucial role in maintaining accurate and consistent inventory data. They ensure that business rules are enforced automatically, and relationships between tables remain valid.

invalid data from being entered and ensure that every product is linked to a valid supplier.

Identification of Key Issues

- Ensure product prices are valid.
- Prevent products from referencing non-existing suppliers.
- Maintain data integrity.
- Demonstrate constraint enforcement.

Relational Schema

Supplier (SupplierID, SupplierName)

Product (ProductID, ProductName, Price, SupplierID)

SQL Statements

```
CREATE TABLE Supplier(
  SupplierID INT PRIMARY KEY,
```

```
SupplierName VARCHAR(50)
);
```

```
CREATE TABLE Product(
ProductID INT PRIMARY KEY,
ProductName VARCHAR(50),
Price DECIMAL(10,2) CHECK (Price > 0),
SupplierID INT,
FOREIGN KEY (SupplierID)
REFERENCES Supplier(SupplierID)
);
```

Constraint Demonstration

```
INSERT INTO Product
VALUES (101,'Mouse',-500,1);
```

Result

CHECK Constraint Violated

```
INSERT INTO Product
VALUES (102,'Keyboard',700,99);
```

Result

FOREIGN KEY Constraint Violated

Explanation

The **CHECK** constraint ensures that product prices are always greater than zero. The **FOREIGN KEY** constraint ensures that a supplier must exist before a product can reference it. If these conditions are violated, the database rejects the insertion.

Advantages

- Maintains data consistency.
- Prevents invalid records.
- Improves database reliability.
- Reduces data entry errors.

CHECK and FOREIGN KEY constraints are essential for maintaining accurate, consistent, and reliable inventory data.

10.Design a relational schema for an HR system and create materialized views for monthly attendance and performance summaries

Scenario Understanding

An HR Management System stores employee information, attendance records, and performance evaluations. The organization wants to create **materialized views** to generate monthly attendance summaries and employee performance reports. Materialized views store precomputed results, making report generation faster and improving database performance.

Identification of Key Issues

- Store employee attendance and performance data.
- Create materialized views for monthly reports.
- Improve query performance.
- Generate reports efficiently.

Relational Schema

Employee (EmpID, EmpName, DeptID)

Attendance (EmpID, AttendanceDate, Status)

Performance (EmpID, ReviewMonth, Rating)

Relationship

- One employee can have many attendance records.
- One employee can have multiple performance reviews.
- Attendance and performance are linked using **EmpID**.

Materialized View – Attendance

```
CREATE MATERIALIZED VIEW MonthlyAttendance AS
SELECT EmpID,
COUNT(*) AS TotalPresent
FROM Attendance
WHERE Status='Present'
GROUP BY EmpID;
```

Materialized View – Performance

```
CREATE MATERIALIZED VIEW MonthlyPerformance AS
SELECT EmpID,
AVG(Rating) AS AverageRating
FROM Performance
GROUP BY EmpID;
```

Explanation

The first materialized view stores the total number of days an employee was present during the

month. The second view stores the average performance rating of each employee. Since the results are stored physically, reports can be generated much faster than executing the queries repeatedly.

Advantages

- Improves report generation speed.
- Reduces query execution time.
- Supports HR decision-making.
- Simplifies monthly reporting.

Materialized views improve the efficiency of the HR Management System by storing summarized attendance and performance information, enabling faster report generation and better workforce management.

